Python –OOP II

Objective

- Inheritance
- super() Function
- Overloading
- Overriding
- Data hiding
- Summary

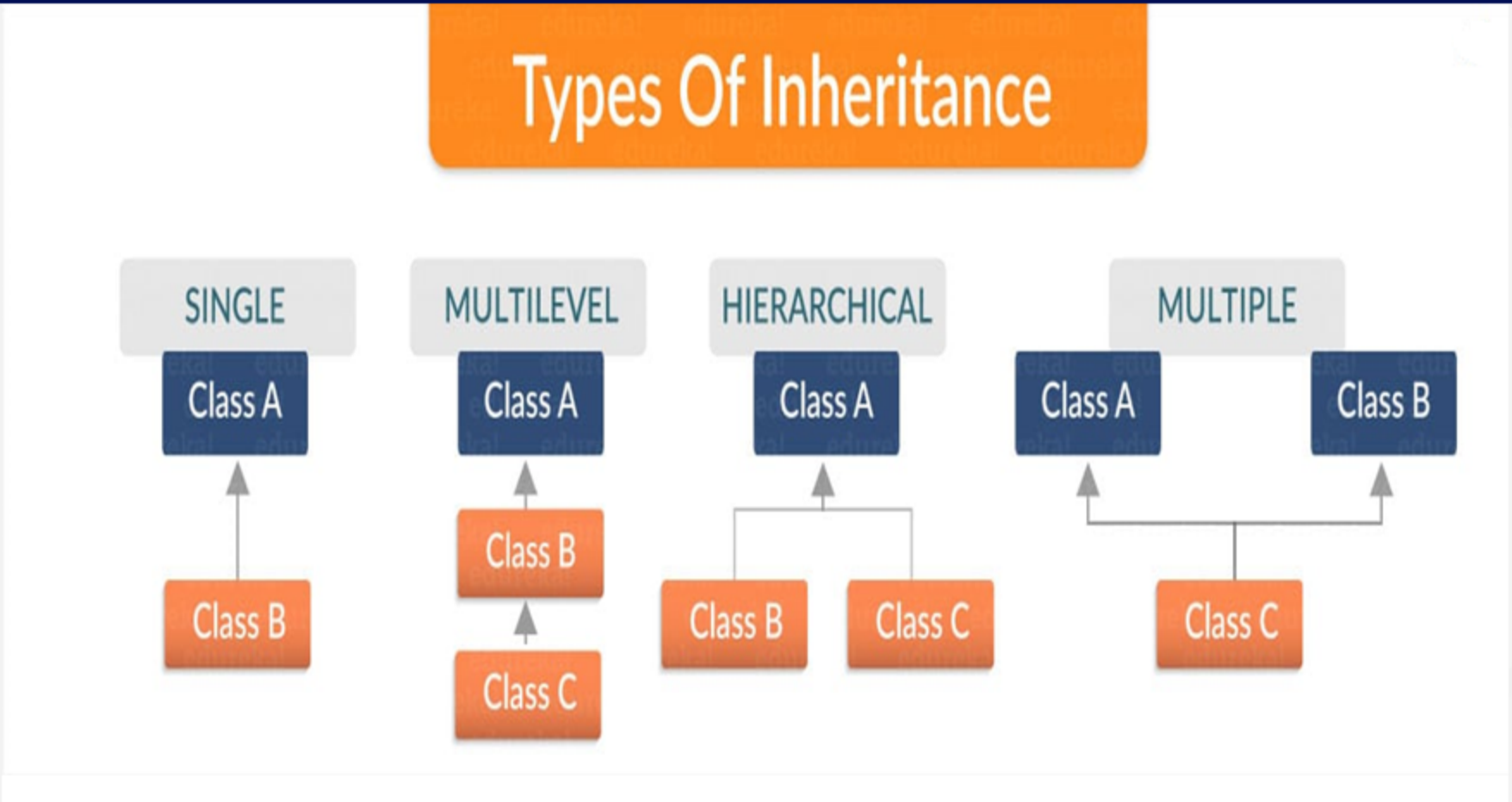
Inheritance

- It specifies that the child object acquires all the properties and behaviors of the parent object.
- By using inheritance, we can create a class which uses all the properties and behavior of another class.
- The new class is known as a derived class or child class or sub class, and the one whose properties are acquired is known as a base class or parent class or super class.
- It provides the re-usability of the code.

```
Syntaxclass derived-class(base class):
        <class-suite>
```


www.object.co.in

Types of Inheritance



__init__( ) Function

The __init__() function is called every time a class is being used to make an object. When we add the __init__() function in a parent class, the child class will no longer be able to inherit the parent class's __init__() function. The child's class __init__() function overrides the parent class's __init__() function.

Function with Inheritance

OBJECT™ TECHNOLOGIES

- Python has two built-in functions that work with inheritance:
- **Use isinstance() to check an instance's type:** isinstance(obj, int) will be True only if obj.__class__ is int or some class derived from int.
- **Use issubclass() to check class inheritance:** issubclass(bool, int) is True since bool is a subclass of int. However, issubclass(float, int) is False since float is not a subclass of int.

- **Sub-classing**
  Calling a constructor of the parent class by mentioning the parent class name in the declaration of the child class is known as sub-classing. A child class identifies its parent class by sub-classing.

- **Python MRO (Method Resolution Order)**

```
1.>>> Child.__mro__
(<class '__main__.Child'>, <class '__main__.Mother'>, <class '__main__.Father'>, <class 'object'>)
```

The __mro__ attribute returns a tuple, but the mro() method returns a python list.

```
1.>>> Child.mro()
[<class '__main__.Child'>, <class '__main__.Mother'>, <class '__main__.Father'>, <class 'object’>]
```

super()

Python super() function provides us the facility to refer to the parent class explicitly. It is basically useful where we have to call superclass functions. It returns the proxy object that allows us to refer parent class by 'super'.

Example 1

```
class Parent():
    def show(self):
        print("Inside Parent")
```

```
class Child(Parent):
    def show(self):
    # Calling the parent's class method
        super().show()
        print("Inside Child")
```

```
# Driver's code
obj = Child()
obj.show()
```

```
Output:
Inside Parent
Inside Child
```


www.object.co.in

Overloading

- Overloading is not supported by Python i.e. you can not have two more methods with same name. But in python we can have single method which can behave like polymorphic (overloaded) method.

Example of inbuilt polymorphic functions :

```
# Python program to demonstrate in-built polymorphic functions
# len() being used for a string
print(len("Hello"))
# len() being used for a list
print(len([10, 20, 30]))
```

Output:
5
3

www.object.co.in

Overriding

- In Python method overriding occurs by simply defining in the child class a method with the same name of a method in the parent class.

```
class Parent(object):
    def __init__(self):
        self.value = 4
    def get_value(self):
        return self.value

class Child(Parent):
    def get_value(self):
        return self.value + 1
c = Child()
c.get_value()
```

Output:

5

www.object.co.in

Data hiding

- Using OOP in Python, we can restrict access to methods and variables.
- This prevents data from direct modification which is called encapsulation.
- In Python, we denote private attributes using underscore as the **prefix double __. protected variables using single _** .

```
Example :class Encapsulation():
   def __init__(self, a, b, c):
      self.public = a
      self._protected = b
      self.__private = c

en =Encapsulation(10,20,30)
print(en.public)
print(en._protected)
print(en.__private)
```

```
Output
10
20
Traceback (most recent call last):
  File "C:/Users/Arnav/Desktop/Python/demo2.py", line 11, in <module>
    print(en.__private)
AttributeError: 'Encapsulation' object has no attribute '__private'
```


www.object.co.in

Summary

- Inheritance
- super() Function
- Overloading
- Overriding
- Data hiding

Thank You